Міністерство освіти і науки України
Вінницький національний технічний університет
Факультет інформаційних технологій та комп'ютерної інженерії
Кафедра програмного забезпечення

Практична робота №5

з дисципліни «Розробка проектів засобами платформи .NET»

Тема: «Основи ASP.NET Core, EF Core та SQLite: перший CRUD Web API»

Виконав: студент гр. 2ПІ-24б
Слободян І. В.

Перевірив:
Позур Михайло Юрійович

м. Вінниця - 2026

GitHub репозиторій: https://github.com/2005igor2023-crypto/dotnetvntu

1. Мета роботи

Навчитися створювати проєкт ASP.NET Core Web API, підключати EF Core з провайдером SQLite, проєктувати першу модель даних та реалізовувати базовий CRUD-контролер з використанням асинхронного програмування.

2. Завдання варіанта №1 («Бібліотека»)

• **Предметна область:** Бібліотека.

• **Модель даних:**

• Author (Id, FirstName, LastName, Country, ICollection<Book> Books);

• Book (Id, Title, ISBN, PublicationYear, Genre, AuthorId, Author);

• Зв'язок «один-до-багатьох» (один Автор $\rightarrow$ багато Книг).

• **Контекст даних:** LibraryDbContext : DbContext з колекціями DbSet<Author> та DbSet<Book>.

• **База даних:** Файлова реляційна база SQLite (Data Source=library_lab05.db).

• **CRUD-контролер:** Асинхронний контролер BooksController (GET all, GET by id, POST, PUT, DELETE).

• **Seed-дані:** Початкове наповнення авторами та книгами при створенні бази.

• **Документація:** Інтеграція OpenAPI та сучасного Scalar UI (/scalar/v1).

3. Структура моделі даних

```
[Author] 1 ——— < [Book]
------------------------
Author (Id, FirstName, LastName, Country)
Book   (Id, Title, ISBN, PublicationYear, Genre, AuthorId [FK])
```

• **Зовнішній ключ:** Book.AuthorId посилається на Author.Id.

• **Каскадне видалення:** При видаленні автора видаляються пов'язані книги (OnDelete(DeleteBehavior.Cascade)).

4. Перелік реалізованих HTTP ендпоінтів

Метод	Маршрут	Опис	Очікувана відповідь
:---:	:---	:---	:---:
`GET`	`/api/books`	Отримати повний список книг разом з автором	`200 OK` (JSON масив)
`GET`	`/api/books/{id}`	Отримати конкретну книгу за Id	`200 OK` / `404 Not Found`
`POST`	`/api/books`	Додати нову книгу до бібліотеки	`201 Created` з Location header
`PUT`	`/api/books/{id}`	Оновити існуючу книгу за Id	`204 No Content` / `400` / `404`
`DELETE`	`/api/books/{id}`	Видалити книгу за Id	`204 No Content` / `404 Not Found`

5. Лістинг контролера (`Controllers/BooksController.cs`)

```
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using LibraryApi.Data;
using LibraryApi.Models;

namespace LibraryApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class BooksController : ControllerBase
{
    private readonly LibraryDbContext _context;

    public BooksController(LibraryDbContext context)
    {
        _context = context;
    }

    [HttpGet]
    public async Task<ActionResult<IEnumerable<Book>>> GetAllBooks()
    {
        return await _context.Books
```

```
            .Include(b => b.Author)
            .AsNoTracking()
            .ToListAsync();
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<Book>> GetBookById(int id)
    {
        var book = await _context.Books
            .Include(b => b.Author)
            .FirstOrDefaultAsync(b => b.Id == id);

        if (book == null) return NotFound(new { message = $"Книгу з Id={id} не знайдено." });
        return Ok(book);
    }

    [HttpPost]
    public async Task<ActionResult<Book>> CreateBook(Book book)
    {
        var authorExists = await _context.Authors.AnyAsync(a => a.Id == book.AuthorId);
        if (!authorExists) return BadRequest(new { message = $"Автора з Id={book.AuthorId} не
існує." });

        _context.Books.Add(book);
        await _context.SaveChangesAsync();

        return CreatedAtAction(nameof(GetBookById), new { id = book.Id }, book);
    }

    [HttpPut("{id}")]
    public async Task<IActionResult> UpdateBook(int id, Book updatedBook)
    {
        if (id != updatedBook.Id) return BadRequest(new { message = "Id не збігається." });

        var existingBook = await _context.Books.FindAsync(id);
        if (existingBook == null) return NotFound(new { message = $"Книгу з Id={id} не знайдено."
});

        existingBook.Title = updatedBook.Title;
        existingBook.ISBN = updatedBook.ISBN;
        existingBook.PublicationYear = updatedBook.PublicationYear;
        existingBook.Genre = updatedBook.Genre;
        existingBook.AuthorId = updatedBook.AuthorId;

        await _context.SaveChangesAsync();
        return NoContent();
    }

    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteBook(int id)
    {
        var book = await _context.Books.FindAsync(id);
        if (book == null) return NotFound(new { message = $"Книгу з Id={id} не знайдено." });

        _context.Books.Remove(book);
        await _context.SaveChangesAsync();
        return NoContent();
```

```
  }
}
```

6. Зразки роботи через Scalar UI / cURL

6.1. Запит `GET /api/books`

```
[
  {
    "id": 1,
    "title": "Кобзар",
    "isbn": "978-966-03-7123-1",
    "publicationYear": 1840,
    "genre": "Поезія",
    "authorId": 1,
    "author": {
      "id": 1,
      "firstName": "Тарас",
      "lastName": "Шевченко",
      "country": "Україна"
    }
  },
  {
    "id": 3,
    "title": "1984",
    "isbn": "978-0-452-28423-4",
    "publicationYear": 1949,
    "genre": "Антиутопія",
    "authorId": 2,
    "author": {
      "id": 2,
      "firstName": "Джордж",
      "lastName": "Орвелл",
      "country": "Велика Британія"
    }
  }
]
```

6.2. Запит `POST /api/books`

```
POST /api/books HTTP/1.1
Content-Type: application/json

{
  "title": "Захар Беркут",
  "isbn": "978-966-03-8888-1",
  "publicationYear": 1883,
  "genre": "Історична повість",
  "authorId": 1
}
```

Відповідь: HTTP/1.1 201 Created

Location: /api/books/5

7. Посилання на репозиторій

• **URL репозиторію:** https://github.com/2005igor2023-crypto/dotnetvntu

• **Папка проєкту:** /05-aspnetcore-efcore-sqlite-crud

8. Відповіді на контрольні питання

1. Що таке Dependency Injection і як `AppDbContext` реєструється у контейнері служб ASP.NET Core?

Відповідь: Dependency Injection (впровадження залежностей) - це архітектурний патерн IoC, при якому об'єкти отримують свої залежності ззовні (через конструктор), а не створюють їх через new. Реєстрація LibraryDbContext виконується методом builder.Services.AddDbContext<LibraryDbContext>(options => options.UseSqlite(...)). Контейнер автоматично інжектує контекст у конструктори контролерів.

2. Навіщо потрібні міграції EF Core і чим Code-First відрізняється від Database-First?

Відповідь: Міграції дозволяють інкрементально еволюціонувати схему бази даних разом зі змінами C#-моделей зі збереженням наявних даних. У підході **Code-First** розробник пише класи мовою C#, а EF Core генерує структуру таблиць БД; у підході **Database-First** спочатку створюються таблиці в СКБД, після чого генеруються C#-класи.

3. Чому методи доступу до бази даних варто робити асинхронними (`async`/`await`)? Що станеться при синхронному виклику під навантаженням?

Відповідь: Операції введення/виведення (I/O) до бази даних є тривалими. Синхронний виклик блокує потік пулу потоків (thread pool) веб-сервера Kestrel. Під навантаженням потік за потоком блокуватимуться, що призведе до вичерпання пулу потоків (thread starvation) і падіння пропускної здатності. Асинхронність (async/await) повертає потік до пулу під час очікування відповіді від диска/мережі, дозволяючи серверу обслуговувати інші запити.

4. Як EF Core визначає зовнішній ключ і навігаційні властивості за конвенцією?

Відповідь: За конвенцією EF Core шукає властивість з назвою <НавігаційнаВластивість>Id або <Ім'яЦільовогоКласу>Id (наприклад, AuthorId для класу Author). Якщо тип збігається з типом первинного ключа цільової сутності (int), EF Core автоматично налаштовує зовнішній ключ (Foreign Key) без необхідності Fluent API чи атрибутів [ForeignKey].

5. Чим відрізняються `AddDbContext` з часом життя Scoped від Singleton і чому для DbContext типово використовують Scoped?

Відповідь: Scoped створює один екземпляр об'єкта на весь життєвий цикл одного HTTP-запиту і знищує його в кінці запиту. Singleton створює один екземпляр на весь час роботи застосунку. DbContext не є потокобезпечним (not thread-safe), тому його категорично не можна робити Singleton - одночасні паралельні HTTP-запити призведуть до стану гонитви та пошкодження кешу відстеження сутностей (change tracker). Режим Scoped забезпечує ізоляцію та безпеку транзакцій.

9. Висновки

У ході виконання практичної роботи №5 було розроблено повноцінний REST-орієнтований Web API застосунок на базі ASP.NET Core (.NET 10) та EF Core з файловою базою даних SQLite для предметної області «Бібліотека». Було спроєктовано зв'язані моделі Author та Book, налаштовано контекст LibraryDbContext, створено та протестовано асинхронний CRUD-контролер BooksController, а також підключено документацію OpenAPI та інтерфейс Scalar UI.